UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

SYSTEM AND NETWORK PROGRAMMING
CEF488

LAB 1: PROCESS MANAGEMENT, SIGNALS AND ADVANCED IPC

GROUP MEMBERS

NAME	MATRICULE	SIGNATURE
NEIL AMBAYI MPOCHE	FE23A105	
EFUETNGONG GENESIS TAZISONG	FE23A041	
TAKU YVETTE WUTOH	FE23A152	
TAMANTENG ABERLINE NGUEMUTANG	FE23A154	

Course Instructor

Mr. FORCHA GLENN

HONESTY REPORT

TASK	RESPONSIBLE MEMBER(S)	CONTRIBUTOR(S)	REVIEWER(S)
Prelab Questions	TAMANTENG	EFUETNGONG	LEADER
Implementation	TAKU	NEIL	LEADER
Testing and debuging	TAKU	NEIL	LEADER
Report writing	TAMANTENG	EFUETNGONG	LEADER

Contents

ABSTRACT

This laboratory exercise focused on advanced operating system concepts, including process management, inter-process communication (IPC), signal handling, daemonization, and non-blocking I/O in Linux systems. The main objective was to design and implement a robust background service capable of creating multiple child processes, enabling bidirectional communication using unnamed pipes, and synchronizing execution through signals.

The system simulated a real-world network monitoring daemon similar to services such as SSH servers, web servers, and monitoring agents. Key system calls used included fork() for process creation, pipe() for IPC, sigaction() for reliable signal handling, and fcntl() for configuring non-blocking communication channels.

The daemonization process involved creating a new session using setsid(), resetting the file mode mask with umask(0), redirecting standard input/output streams, and detaching from the controlling terminal. Additionally, graceful termination and cleanup mechanisms were implemented using signals such as SIGTERM, SIGINT, and SIGCHLD.

The results demonstrated that the daemon operated independently in the background, handled child processes correctly, and maintained efficient, non-blocking communication. Overall, the lab provided practical experience in concurrent processing, asynchronous communication, and Linux system-level programming.

OBJECTIVES

The main objectives of this laboratory exercise were:

- To understand process creation and management in Linux systems
- To implement daemon processes
- To study process groups and sessions
- To implement inter-process communication using unnamed pipes
- To explore signal handling mechanisms
- To implement non-blocking I/O operations
- To manage child process synchronization

PRE-LAB ANSWERS

1. What is the difference between a process group and a session? Why must a daemon create a new session?

   A process group is a collection of related processes that can receive signals together. Each group has a unique Process Group ID (PGID).

   A session is a higher-level structure that contains one or more process groups and is usually associated with a terminal. The first process in a session is called the session leader.

   Why a daemon must create a new session:

A daemon uses setsid() to:

- Detach from the controlling terminal
- Avoid receiving terminal-generated signals (e.g., SIGINT)
- Run independently in the background

Without this step, the daemon may terminate when the user logs out.

2. List at least three steps required to daemonize a process and explain why each is needed.

- Step 1: Fork and Exit Parent (fork();)
  Ensures the child process is not a process group leader.
- Step 2: Create a New Session (setsid();)
  Detaches the process from the terminal.
- Step 3: Change Working Directory (chdir("/");)
  Prevents locking of mounted file systems.
- Step 4: Reset File Creation Mask (umask(0);)
  Ensures predictable file permissions.

3. Describe the difference between sigaction() and signal(). Why is sigaction() preferred?

- signal() is older and less reliable
- sigaction() is modern and POSIX-compliant

Why sigaction() is preferred
Sigaction() is preferred because of the following reasons:

- More reliable and consistent
- Provides better control
- Prevents race conditions
- Portable across systems

4. What is the purpose of the SA RESTART flag in sigaction()?

The SA_RESTART flag ensures that interrupted system calls are automatically restarted after a signal handler completes.

5. Why is it unsafe to call printf() inside a signal handler? Give an example of a safe alternative.

Printf() is not safe because it is not async-signal-safe and may use shared resources like buffers.

6. Explain how two unnamed pipes can be used for bidirectional communication between a parent and a child.

A single pipe supports only one-way communication. For two-way communication:

- Create two pipes
- One for Parent → Child

- One for Child → Parent

This enables full-duplex communication.

7. What is the Significance of PIPE_BUF? How does it affect write atomicity?

PIPE_BUF defines the maximum size of data that can be written atomically to a pipe.

This ensures:

- No data mixing between processes
- Reliable communication

8. Describe how fcntl(fd, F_SETFL, O NONBLOCK) changes the behavior of read () and write ():

fcntl(fd, F_SETFL, O_NONBLOCK);

- read() returns immediately if no data is available
- write() does not block if the buffer is full

This improves responsiveness.

9. What system call would you use to limit the number of processes a user can create? What structure does it fill?

The system call used is:

setrlimit()

It uses the structure:

struct rlimit

This limits system resources such as:

- Number of processes
- Memory usage
- CPU time

METHODOLOGY

SYSTEM DESIGN OVERVIEW

The system was designed as a daemon that spawns multiple child processes. Each child simulates a network monitoring agent. Communication between the daemon and child processes was achieved using unnamed pipes.

Implementation stages:

- Process creation using fork()
- Daemonization
- Pipe creation for IPC

- Non-blocking I/O configuration
- Signal handling setup
- Child process synchronization and cleanup

Process Creation

The fork() system call creates a new child process:

- The child is a copy of the parent
- Each process has a unique PID
- Both execute independently

Daemonization Procedure

Steps implemented:

- ✓ Fork and exit parent
- ✓ Create new session using setsid()
- ✓ Change working directory to root
- ✓ Reset file mask using umask(0)
- ✓ Close all open file descriptors
- ✓ Redirect standard input/output to /dev/null

Inter-Process Communication (IPC)

Two pipes were created:

```
int pipe1[2];
int pipe2[2];

pipe(pipe1);
pipe(pipe2);
```

- Pipe 1: Parent → Child
- Pipe 2: Child → Parent

This enabled bidirectional communication.

Non-Blocking I/O

The read ends of the pipes were configured as non-blocking, allowing continuous execution without waiting for input.

Signal Handling

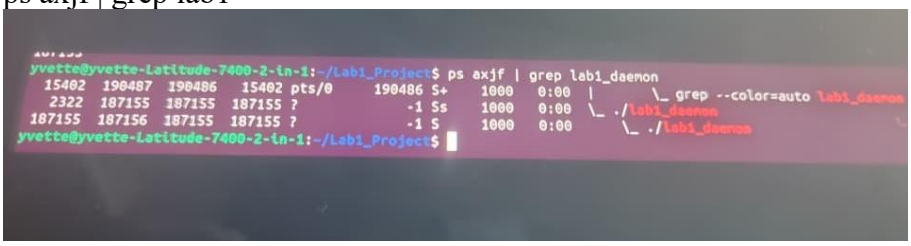
Signals used:

- SIGCHLD → Detect child termination
- SIGTERM → Graceful shutdown
- SIGINT → Interrupt handling

Expected Output and Verification

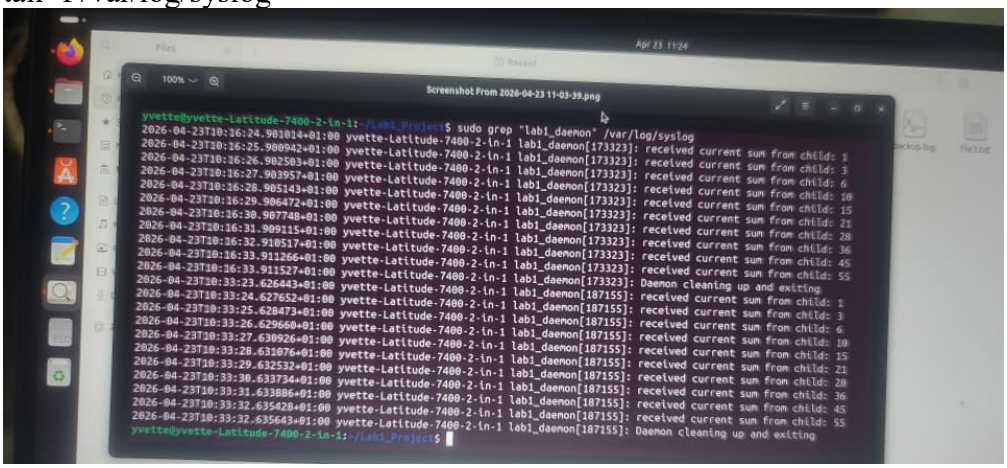
- Running ./lab1 daemon starts the daemon in the background
- PID is stored in /tmp/mydaemon.pid
- Verified using:

ps axjf | grep lab1



- Logs monitored using:

tail -f /var/log/syslog



- Termination:

kill -TERM $(cat /tmp/mydaemon.pid)

RESULTS OBTAINED

1. Successful Daemon Creation

- Detached from terminal
- Continued running after logout

2. Child Process Creation

- Multiple children created
- Each operated independently

3. Bidirectional Communication

- Messages exchanged successfully

4. Non-Blocking I/O

- No system freezing
- Immediate read response
- Continuous monitoring

ANALYSIS AND DISCUSSION

Importance of Daemonization

Daemonization allows services to run continuously without user interaction. The implemented daemon behaved like real Linux services such as web servers and SSH daemons.

Efficiency of Pipes

Advantages:

- Fast
- Simple
- Reliable

Limitations:

- Only works for related processes
- Requires two pipes for bidirectional communication

Benefits of Non-Blocking I/O

- Improves responsiveness
- Prevents system blocking
- Enables multitasking

Widely used in:

- Web servers
- Databases
- Real-time systems

Role of Signal Handling

Signals enable asynchronous control of processes.

Using sigaction() provided:

- Reliable handling
- Better control
- Prevention of race conditions

Proper handling of SIGCHLD avoided zombie processes.

Process Synchronization Challenges

Challenges encountered:

- Deadlocks
- Unexpected termination
- Coordination of IPC

Solutions:

- Signal-based synchronization
- Non-blocking I/O
- Proper cleanup

Resource Management

Using setrlimit() improved:

- Stability
- Security
- Fault tolerance

CONCLUSION

This laboratory successfully demonstrated advanced Linux process management and IPC techniques. A functional daemon was implemented that created and managed multiple child processes, communicated using unnamed pipes, and handled signals efficiently.

Key concepts such as daemonization, non-blocking I/O, signal handling, and process synchronization were clearly demonstrated. The use of system calls like fork(), pipe(), sigaction(), and fcntl() provided practical understanding of concurrent process management.

The experiment highlighted how real-world Linux services operate efficiently in the background while maintaining responsiveness and proper resource management.